StudyForge Stage 2 Implementation Report

Course: EECS3311 Software Design, Fall 2026

Repository: ZaaidAhmed/StudyForge

Documentation revision: `f7995f6` (November 10, 2026)

Application verification revision: `fa47c4b` (Java 21 CI and clean-clone checks)

Status: Application verification is recorded for `fa47c4b`; later documentation changes are present at `f7995f6`. The report PDF is the authoritative report deliverable. It has no retained Markdown source. Provider billing exports and instructor-access confirmation were not available for this report.

1. Project overview and implementation status

StudyForge is a local-first learning workspace for students. It imports course PDFs, text, and Markdown, stores normalized source chunks, provides explainable lexical retrieval, and supports grounded learning activities. A Spring MVC/Thymeleaf GUI and Picocli CLI use the shared `StudyAgentFacade` and application services. Spring Data JPA and file-backed H2 store course and learning data. The provider-neutral `LlmClient` routes to local Ollama or hosted Gemini. The agent performs intent analysis, bounded planning, allowlisted course-scoped tool execution, recent-history retrieval, and response synthesis; model proposals are validated before tools run.

The Stage 1 proposal contains F01-F14. All fourteen are implemented in the current codebase, with differences documented in Section 4 and the crosswalk. F15, runtime selection of local/cloud AI providers, is a Stage 2 addition. The codebase has working GUI and CLI paths for the commands listed in the README; F09 quiz taking/grading and F10 wrong-answer explanations are intentionally GUI-only. This report distinguishes code support from feature-specific public video evidence.

Known limitations include no OCR for scanned/image-only PDFs, no semantic/vector retrieval, local single-user operation without authentication, and provider-dependent generation quality/latency. These are outside the proposal's initial scope. Stage 2 requires basic verification; the full deterministic test plan and KUMA behavioral evaluation are Stage 3 work.

2. Feature-to-implementation mapping

Every Stage 1 proposal feature is retained in the mapping. GUI and CLI describe the implemented interfaces; where an interface is intentionally absent, it is stated. Stage 1 use-case and sequence identifiers refer to the proposal's UC01-UC14 and SD01-SD12 diagrams. Each feature links to its standalone video; useful in-video CLI segments have separate jump links. The full index is docs/stage2/demo-index.md.

Feature	Stage 1 reference; status/type	Implemented classes and responsibilities; key operations	Patterns	Verification and demo
F01 Course workspace	UC01, SD11; implemented; deterministic	`CourseController` handles GUI requests; `CourseService` validates and manages courses (`createCourse`, `getAllCourses`, `getCourseByCode`, `updateCourse`, `deleteCourse`); `CourseRepository` persists; Picocli course commands expose supported create/list/show operations.	MVC, Facade, Command, Repository	
F02 Material import and processing	UC02, SD04/SD11; implemented; deterministic	`MaterialController` and `MaterialAddCommand` accept files; `CourseMaterialService.importMaterial` validates course/type/size/hash, calls storage/extractor/normalizer/chunker collaborators, and persists `CourseDocument`/`DocumentChunk`.	MVC, Strategy-like extractor polymorphism, Facade, Command, Repository	
F03 Course material retrieval	UC03, SD11; implemented; deterministic	`CourseSearchService.search` scopes READY chunks to the course; `QueryNormalizer` and `LexicalRetrievalScorer` rank them; `MaterialController`/`MaterialSearchCommand` present source-aware results.	MVC, Facade, Command, Repository	
F04 Grounded course Q&A	UC04, SD01; implemented; hybrid	`GroundedQuestionService.answer` retrieves through `CourseSearchService`, creates bounded evidence with `GroundingContextBuilder`, builds a prompt, calls `LlmClient`, parses structured output, and validates source IDs; `AskController`/`AskCommand` expose it.	MVC, Facade, Adapter, Command	
F05 Document/topic summaries	UC05, SD05; implemented; hybrid	`SummarizationService.summarizeDocument` and `summarizeTopic` validate scope, assemble bounded evidence, call the provider, parse and validate cited sources; `LearningController`/`SummaryCommand` present results.	MVC, Facade, Adapter, Command	
F06 Key concept extraction	UC06, SD06; implemented; hybrid	`ConceptExtractionService.extract` requests structured concepts, validates source IDs, and deterministically merges duplicates; `LearningController`/`ConceptsCommand` expose extraction.	MVC, Facade, Adapter, Command	
F07 Flashcard generation/review	UC07, SD07; implemented; hybrid generation and deterministic review	`FlashcardGenerationService.generate` validates document scope, extracts concepts, generates/retries/backfills, validates/deduplicates and persists cards/sources; `FlashcardReviewService.review` validates ownership/rating and appends review records; web and CLI commands expose supported operations.	MVC, Facade, Adapter, Command, Repository	
F08 Grounded quiz generation	UC08, SD08; implemented; hybrid	`QuizGenerationService.generate` builds document-grounded questions, parses and validates type/options/sources, rejects duplicates, recovers where possible, and persists the quiz; `QuizController`/`QuizCommand` provide GUI and CLI generation/listing.	MVC, Facade, Adapter, Command, Repository	
F09 Quiz taking/grading	UC09, SD09; implemented; hybrid for legacy short answers	`QuizAttemptService.submit` validates quiz/question ownership, grades objective items deterministically, uses source-checked model grading for legacy short answers with deterministic fallback, and persists `QuizAttempt`/`QuizResponse`; GUI only.	MVC, Facade, Repository	
F10 Evidence-based wrong-answer explanations	UC10, SD10; implemented; hybrid	`WrongAnswerExplanationService.explain` validates course/quiz/question/attempt, uses the persisted answer and question sources, validates the response/citations, and leaves the grade unchanged; GUI only.	MVC, Facade, Adapter, Repository	
F11 Progress/weak-topic analysis	UC11, SD12; implemented; deterministic	`ProgressService.summarize` aggregates persisted flashcard ratings and quiz attempts into overall and topic scores/status; progress controller, agent tools, and `ProgressCommand` present it.	MVC, Facade, Command, Repository	
F12 Personalized study plans	UC12, SD03; implemented; deterministic	`StudyPlanService.create` validates dates/budget, selects `StudyPlanningStrategy`, enforces allocation constraints, persists plan/items; balanced, weak-topic-focused, and exam-cram strategies implement distinct allocations; GUI/CLI supported.	MVC, Facade, Strategy, Command, Repository	
F13 Recommendations and bounded agent assistance	UC13, SD02; implemented; hybrid	`RecommendationService.recommend` selects a deterministic next action; `AgentCoordinator` uses `IntentAnalyzer`, `TaskPlanner`, `AgentPlanValidator`, `AgentToolManager`, `MemoryManager`, and `ResponseSynthesizer` for bounded, course-scoped tasks; `RecommendationCommand`/`AgentCommand` expose CLI paths.	MVC, Facade, Adapter, Command	
F14 Learning history/bounded memory	UC14, SD12; implemented; deterministic	`LearningHistoryService` derives newest-first events from persisted flashcard reviews, quiz attempts, and study plans, capped at 100; `MemoryManager.recent` supplies ten recent events to synthesis; `HistoryController`/`HistoryCommand` expose history.	MVC, Facade, Command, Repository	
F15 Local/cloud AI provider selection	New in Stage 2; implemented; deterministic configuration/routing	`AiStatusController` validates GUI provider/model selection; `AiProperties` stores active runtime configuration; `RoutingLlmClient` delegates to `OllamaLlmAdapter` or `GeminiLlmAdapter`; CLI selects via environment and `AiStatusCommand` reports the active provider/model. No dedicated CLI switch command exists.	MVC, Adapter, Facade, Command	

The table records representative tests rather than claiming that each listed class has exhaustive coverage. The final clean test result is in Section 6.

3. Implemented design patterns

The following five patterns are distinct and visible in code. Repository is also used as a persistence pattern, but is not needed to meet the five-pattern minimum.

Model-View-Controller (MVC)

Problem: Separate HTTP handling and HTML rendering from reusable course behavior. **Participants/roles:** Spring MVC controllers parse requests and choose views; application services execute use cases; Thymeleaf templates render state. **Evidence:** `web/Controller.java`, `application/service/`, and `src/main/resources/templates/`. **Stage 1:** Matches the proposal. It allows the GUI and CLI to use the same service rules.

Facade

Problem: Give the GUI, CLI, and bounded agent a stable application boundary instead of exposing collaborator graphs. **Participants/roles:** `StudyAgentFacade` delegates use-case calls to application services and `AgentCoordinator`; MVC controllers and Picocli command objects are clients. **Evidence:** `StudyAgentFacade` operations including `askQuestion`, generation/listing, grading, progress, and `agentAssist`. **Stage 1:** Matches the proposed shared facade.

Adapter

Problem: Keep provider HTTP protocols out of feature services. **Participants/roles:** `LlmClient` is the provider-neutral port; `OllamaLlmAdapter` and `GeminiLlmAdapter` translate provider requests/responses; `RoutingLlmClient` selects the configured adapter. **Evidence:** `application/ai/LlmClient`, `infrastructure/ai/*LlmAdapter`, and `RoutingLlmClient`. **Stage 1:** Preserves the planned Ollama Adapter and extends it with Gemini and runtime routing as F15.

Strategy

Problem: Vary study allocation without embedding every policy in one scheduler. **Participants/roles:** `StudyPlanningStrategy` defines the policy contract; `BalancedStudyStrategy`, `WeakTopicFocusedStrategy`, and `ExamCramStrategy` produce activities; `StudyPlanService` validates constraints and persists. **Evidence:** `StudyPlanService.create` and strategy `build` methods. **Stage 1:** Matches the proposal's strategy-based scheduling.

Command

Problem: Represent supported CLI actions as independently parsed operations and keep argument handling out of services. **Participants/roles:** Picocli is the invoker; concrete command classes such as `CourseCreateCommand`, `MaterialSearchCommand`, `QuizCommand.Generate`, `RecommendationCommand`, and `AgentCommand` hold arguments and invoke `StudyAgentFacade`/services. **Evidence:** `cli/*Command.java` and each command's `run`. **Stage 1:** Matches the proposed Picocli Command pattern.

Supplementary Repository pattern: Spring Data repositories isolate persistence for course, documents/chunks, flashcards/reviews, quizzes/attempts, and study plans. This supports testability and scoped queries; it is documented as supplementary because the five patterns above already satisfy the required count.

4. Changes from the Stage 1 design and rationale

Original design	Implemented change	Reason	Impact
F01 course operations and shared facade	Separate CLI Spring entry point/profile; course lookup uses `getCourseByCode` and DTO boundaries.	Avoid starting the web server for CLI use and clarify the service contract.	Same course behavior and shared application logic; no feature removed.
F02 `MaterialImportService`/`FileStorage` names	`CourseMaterialService`/`FileStorageService`; extractor implementations selected by type.	Align names with the existing service convention and isolate format-specific extraction.	Import validation, safe storage, extraction, normalization, chunking, and persistence remain traceable to UC02.
F03 `MaterialSearchService`	`CourseSearchService` with deterministic lexical ranking behind `RetrievalScorer`.	Clarify course scope and keep retrieval explainable; embeddings/vector search were excluded in Stage 1.	Search remains deterministic and source-addressable; no semantic search is claimed.
Stage 1 proposed local Ollama `qwen2.5-coder:14b` provider	Provider-neutral `LlmClient` routes to Ollama `qwen3:4b` or Gemini; CLI selection uses process environment; F15 GUI selection is new.	Fit local hardware and allow a hosted option while preserving provider isolation.	Adds F15; API credentials remain local configuration. Exact deployment/configuration behavior is described in README.
Quiz and flashcard generation used bounded single/limited retry contracts	Generation now includes structured parsing, quality/source validation, semantic-ish deduplication, retry/backfill, and partial-result reporting.	Observed provider schema variance, noisy source text, and duplicate/short generation outputs.	Safer persisted content and clearer shortfall reporting; it cannot guarantee a target count when unique grounded content is exhausted.
Stage 1 browser document view	Application-controlled page rendering for document chat.	Keep document Q&A inside the StudyForge workspace and control navigation.	Preserves selected-document Q&A; no browser PDF embedding is required.
Quiz deletion without explicit dependent cleanup	Course-scoped deletion removes dependent attempts before the quiz.	Prevent orphaned attempt rows.	Deletion remains scoped and consistent with persistence constraints.
Stage 1 F14 bounded recent memory	History page groups/filter activities; memory remains ten recent events and is passed selectively to relevant agent synthesis.	Make persisted activity easier to inspect while keeping memory bounded and relevant.	History remains derived only from flashcard reviews, quiz attempts, and plans; no transcripts or hidden reasoning are stored.

The Stage 1 proposal remains the unmodified baseline. Updated class, use-case, and sequence diagrams are linked in Sections 8 and 9.

5. AI tools and human collaboration

The student used AI in two distinct ways: AI-assisted software development and AI services inside the finished application. The retained collaboration log records representative interactions and human review. It identifies GPT-5 for Phases 1-3; exact model names and account-level usage for later development sessions were not retained.

Codex development assistance. Codex supported architecture discussion, implementation, debugging, tests, user-interface iteration, and documentation. The student set feature scope and acceptance criteria, reviewed outputs, supplied screenshots and observed failures, and directed fixes. The student reports a monthly subscription cost of approximately **$28**. No project-specific token totals are recorded.

Ollama application runtime. Local models documented in the project include `qwen3:4b` and `qwen-claude-14b:latest`. They support course-grounded answers and generation. The application validates model output before persistence. Ollama does not create a hosted API charge; electricity and hardware costs were not measured, and the application does not retain token totals.

Gemini application runtime. Gemini is an optional hosted provider selected through configuration. The API key stays in local environment configuration and is excluded from the report. No authoritative provider usage or billing export was available, so Gemini token and monetary totals are unknown.

The reported subscription price is separated from application inference costs. Missing telemetry is not treated as zero, and no project-level token or cost estimate is claimed. See the retained AI collaboration log for representative tasks and review outcomes.

6. Development and verification summary

Verification performed for this report

- **Verification revision:** `fa47c4b`, checked November 7, 2026, using OpenJDK 21.0.12.1 and Maven Daemon 1.0.6. The current documentation revision is `f7995f6` (November 10, 2026).
- **Automated suite:** `mvnd clean test` completed successfully: **207 tests, 0 failures, 0 errors, 0 skipped**.
- **Package:** `mvnd package` completed successfully on Java 21 on November 7, 2026; it reran the suite with **207 tests, 0 failures, 0 errors, 0 skipped** and produced `target/studyforge-ai-0.1.0-SNAPSHOT.jar`.
- **GitHub Actions:** the cross-platform `Verify` workflow passed on the published commit `fa47c4b` (run 42416941967, Ubuntu, Java 21): **207 tests, 0 failures, 0 errors, 0 skipped**, package and fresh-JAR smoke passed. The earlier red checks were caused by a Unix-host filename test gap; upload validation now rejects both slash styles on every operating system.
- **Fresh-JAR web smoke:** `scripts/verify.ps1 -SkipBuild -Port 18080` started the packaged JAR and received HTTP 200 from `/`, `/courses`, and `/ai/status`.
- **CLI checks:** packaged-JAR help commands for `quiz`, `recommend`, and `agent` completed. Read-only `quiz list --course EECS3221`, `recommend --course EECS3221`, and `history --course EECS3221` also completed. This confirms command registration and these read workflows, not a full AI-backed quiz generation or agent task run.
- **Clean-clone verification:** the published `fa47c4b` was cloned into an isolated temporary directory on Java 21. `mvnd clean package` passed with **207 tests, 0 failures, 0 errors, 0 skipped**; `scripts/verify.ps1 -SkipBuild` returned HTTP 200 for `/`, `/courses`, and `/ai/status`. README-style `sf --help`, `sf course list`, and `sf material list` commands passed using an isolated H2 database.
- **Course isolation and persistence:** in that isolated database, two verification courses were created; course A remained available to a later CLI process after the creating process exited. A document imported to course A did not appear in course B's list, and `material show` for that document under course B returned not found without revealing its content.
- **Provider and live AI generation:** the test suite uses controlled clients for repeatability; a passing test suite does not establish that every external provider or generated answer is available/valid at submission time.
- **Browser and CLI demonstration:** use the public video index and README for feature invocations. F09/F10 are GUI-only; F15 CLI behavior is environment selection followed by `ai status`.

The tests cover deterministic services, parsing, retrieval, repository behavior, controllers/templates, CLI command registration, and AI adapters. Stage 2 requires basic verification; Stage 3 calls for broader deterministic coverage and KUMA behavioral evaluation. No KUMA result is claimed here.

Repository and release evidence

The weekly summary lists five meaningful code commits in each reporting period (October 20-26, October 27-November 2, and November 3-9, Toronto local dates). Unauthenticated HTTP HEAD checks on November 7 returned status 200 for all 15 commit links. In a clean clone, tracked-path checks found no `.env`, local databases,

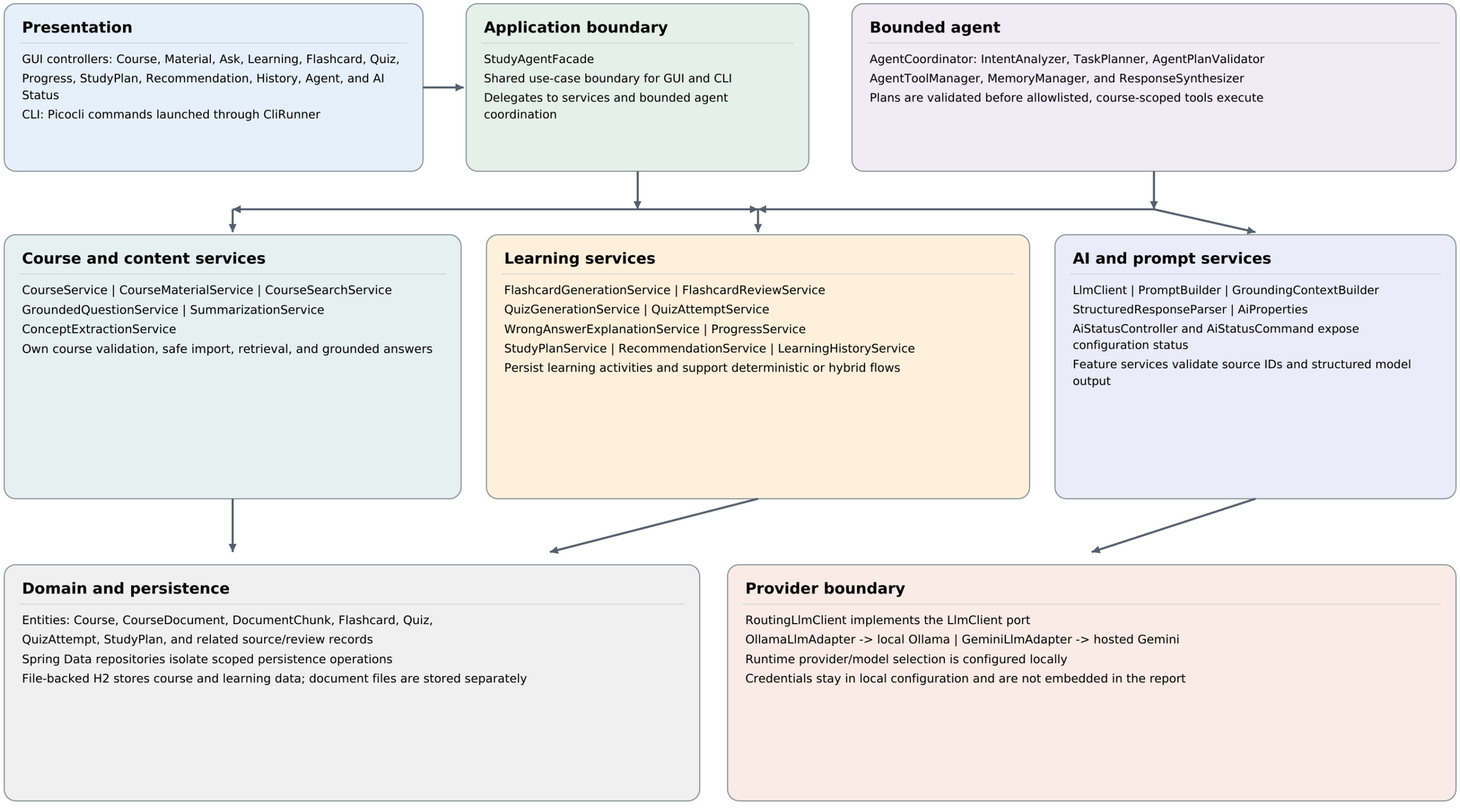
uploads, logs, build output, local demo index, or `unsupported.exe`; a scan for common Gemini, GitHub, and OpenAI key formats found no matches. `.gitignore` excludes `.env`, local data/uploads, logs, build output, and local config; `.env.example` contains placeholders, not a credential. Instructor access to the submission remains unconfirmed.

7. Demonstration evidence

The table below maps every feature, F01-F15, to its standalone recording and matches the public demonstration index. All recordings and demonstrated workflows have been reviewed and confirmed. Each video link opens the recording directly; jump links are included for the useful CLI segments in F08, F13, and F15. F09/F10 are GUI-only. F15 selects its CLI provider through environment configuration and verifies it with `ai status`; there is no provider-switch subcommand.

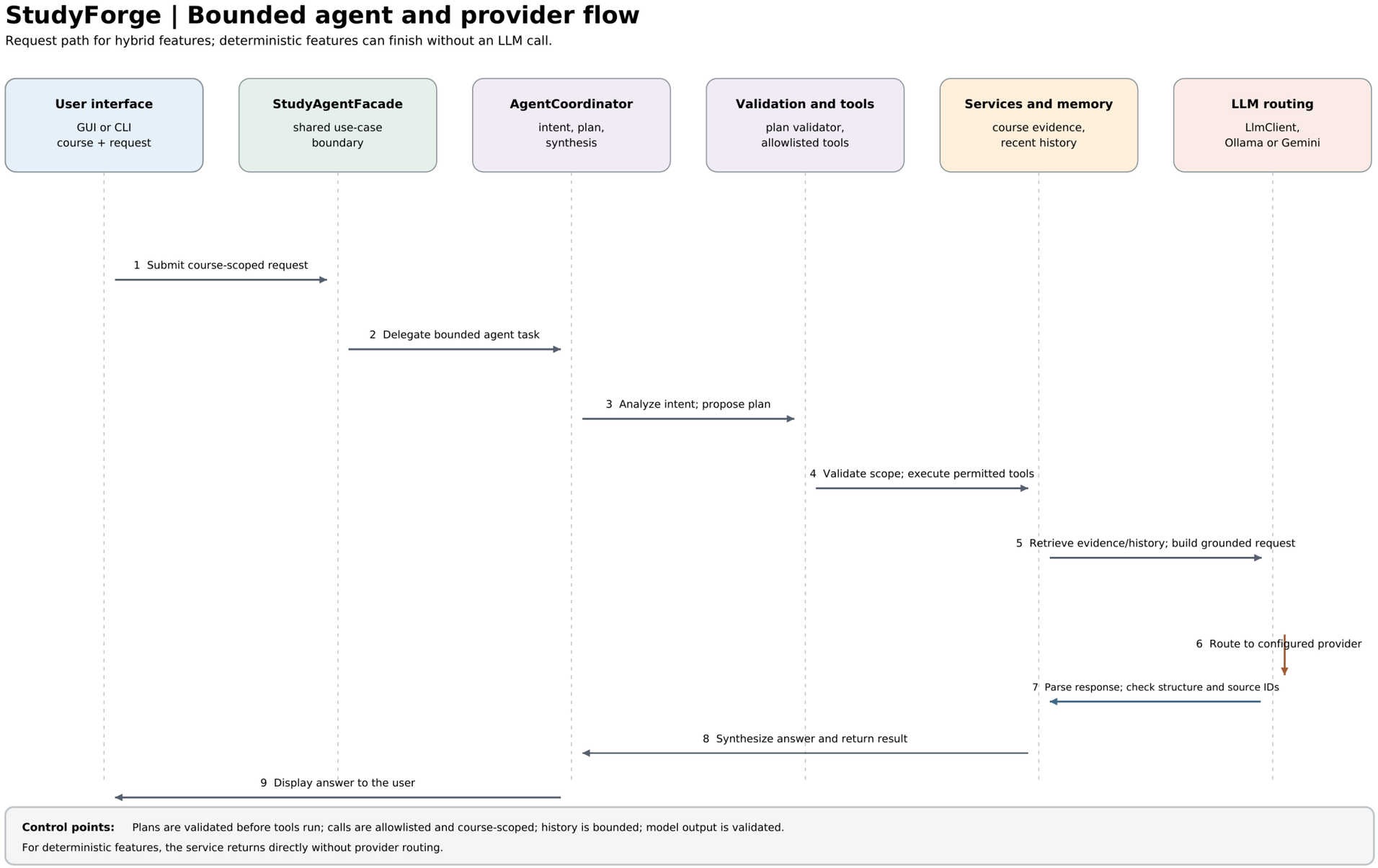
Feature	Recording	Feature evidence	CLI evidence and verification
F01 Course workspace	Watch video	Course workspace video	`course create/list/show`; CLI verified; no jump link listed
F02 Material import	Watch video	Material import video	`material add/list/show`; CLI verified; no jump link listed
F03 Material search	Watch video	Course material search video	`material search`; CLI verified; no jump link listed
F04 Grounded Q&A	Watch video	Grounded Q&A video	`ask`; CLI verified; no jump link listed
F05 Summaries	Watch video	Summary generation video	`summary document/topic`; CLI verified; no jump link listed
F06 Concepts	Watch video	Concept extraction video	`concepts extract`; CLI verified; no jump link listed
F07 Flashcards	Watch video	Flashcard generation and review video	`flashcards generate/list/review`; CLI verified; no jump link listed
F08 Quiz generation	Watch video	Quiz generation video	`quiz generate/list` 01:35
F09 Quiz taking and grading	Watch video	GUI quiz attempt and grading; GUI-only feature	GUI-only; no CLI workflow
F10 Wrong-answer explanation	Watch video	GUI wrong-answer explanation; GUI-only feature	GUI-only; no CLI workflow
F11 Progress	Watch video	Progress and weak-topic video	`progress`; CLI verified; no jump link listed
F12 Study plan	Watch video	Study-plan video	`study-plan create/list`; CLI verified; no jump link listed
F13 Recommendations and agent assistance	Watch video	Recommendation and bounded-agent video	`recommend` 03:05; `agent` 03:25, 03:41, 04:09
F14 Learning history and bounded memory	Watch video	Learning history and memory video	`history`; CLI verified; no jump link listed
F15 Local/cloud AI provider selection (new)	Watch video	Provider and model configuration video	Environment-based Ollama plus `ai status` 01:48; Gemini plus `ai status` 02:53. No provider-switch CLI subcommand.

8. Implemented architecture overview



System layers and their responsibilities. The detailed UML class model is available in the repository PlantUML source.

9. Bounded agent and provider flow



Course-scoped request validation, tool execution, evidence retrieval, provider routing, and response synthesis. The detailed UML class model is available in the repository PlantUML source.

10. Weekly implementation commits

The repository records five meaningful implementation commits for each reporting period. The links below open the published commit details; documentation-only changes are excluded from these weekly counts. The complete weekly summary provides the period overview.

October 20-26, 2026

Implementation	Commit
Course workspace	3343f8f
Grounded search and Q&A	845661d
Material processing	84d2487
Course management	1a4ec16
Retrieval scorer	dfd68ab

October 27-November 2, 2026

Implementation	Commit
Provider routing	2ad0e81
Structured generation	c952c28
Grounded study workflows	eb2ba2b
Document flashcard decks	0d88ef0
Quiz attempts and insights	01517a5

November 3-9, 2026

Implementation	Commit
AI hardening	1b1f285
History timeline	c32bc35
Study-plan guidance	297f10f
Agent workflows	9b0d17d
Verification tooling	5a91a8d